# Digital Professor Integration Plan

Architecture, Course Knowledge, and Interaction Evidence

October 2026

## Contents


**1 Purpose and current priority** **2**

**2 High-level architecture** **2**
2.1 Course preparation and downstream consumers . . . . . 2
2.2 Tutoring-turn flow . . . . . 3

**3 Module responsibilities** **3**
3.1 Structured Course Context and Question Schema . . . . . 5
3.2 Main logical functions . . . . . 5

**4 Key inputs and outputs** **7**

**5 Handoffs between components** **8**

**6 Supporting note: document extraction and RAG design** **10**
6.1 Ingestion pipeline . . . . . 10
6.2 Chunking strategy . . . . . 10
6.3 Setting up retrieval for success . . . . . 11

**7 Supporting note: persistent data and database schemas** **11**

**8 Complete workflow: what enters, leaves, and is tracked** **13**
8.1 Course setup workflow . . . . . 13
8.2 Tutoring-turn workflow . . . . . 14
8.3 Learner state to tutoring action . . . . . 14

**9 Things to agree on** **15**

**10 Things to do and proposed collaboration** **16**

**11 Future work** **16**

# 1 Purpose and current priority

The immediate goal is to align the team on the Digital Professor's overall architecture, module boundaries, responsibilities, inputs and outputs, persistent data, and handoffs. The system remains course-agnostic. Course-specific tutoring workflows and validation sets come later, after the component contracts are stable.

Two connected paths are central to the design:

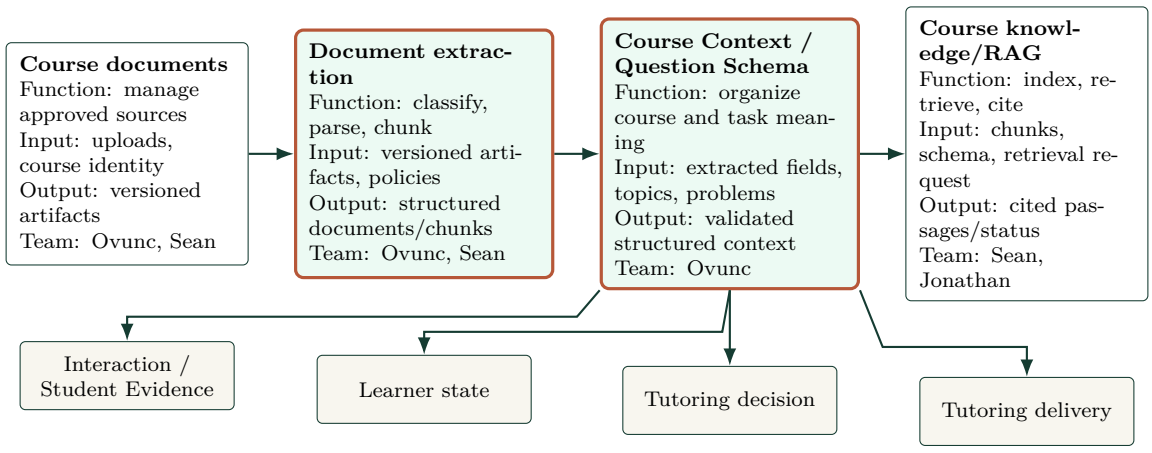
- **Document extraction, structured Course Context / Question Schema, and RAG** convert approved material into both verified system-wide structure and searchable course knowledge.
- **Interaction and Student Evidence** convert student input into a traceable record, retrieve relevant course context, coordinate tutoring decisions, deliver an action, and capture the student's response.

This phase should produce a component architecture, clear ownership boundaries, agreed handoff records, database schemas that support ingestion and retrieval, and a complete course-agnostic workflow. It should not commit the team to a particular framework, model provider, or deployment architecture.

# 2 High-level architecture

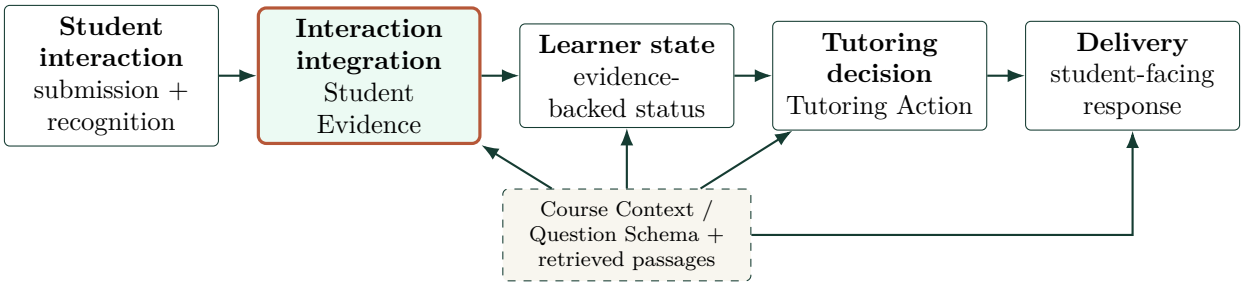
The architecture has two connected flows. The first prepares both searchable course material and structured course/question context. The second uses those outputs to interpret a student turn, update learner state, select a tutoring action, and deliver it.

## 2.1 Course preparation and downstream consumers




The structured context is not merely metadata for retrieval. It gives downstream modules a shared definition of the course, concept, question, expected response, prerequisites, source references, and assessment constraints. RAG supplies passages; the structured schema supplies stable meaning and relationships.

### 2.2 Tutoring-turn flow




The state-to-action boundary is explicit: learner state reports what the evidence supports; tutoring decision applies the question schema, course constraints, and tutoring policy to choose what happens next. Delivery may use the same schema to render the problem, rubric-aware prompts, response controls, citations, or avatar behavior without changing the selected action.

## 3 Module responsibilities

Each module owns one part of the system. The boundary statements prevent two modules from silently making the same decision.

| Module | Responsibility and boundary |
|---|---|
| Course documents | Preserve approved syllabi, lecture notes, textbooks, problems, and other source files with stable identities and versions. This module owns original course artifacts, not extracted interpretations. |
| Document extraction | Register and classify sources; recover pages, structure, equations, tables, and course fields; create traceable chunks. It reports uncertainty rather than inventing missing content. |
| Course Context / Question Schema | Convert verified course structure and authored problems into stable, reusable records. It owns course, topic, prerequisite, question, expected-response, rubric, and source-reference meaning; it does not retrieve passages or judge a student's response. |
| Course knowledge/RAG | Index extracted chunks and retrieve ranked, approved course passages with page citations. It supplies grounding but does not diagnose the student or choose a tutoring action. |
| Student interaction | Capture and recognize text, handwriting, speech, images, equations, and files while preserving the original submission and asking for confirmation when meaning is ambiguous. |
| Interaction integration | Construct Student Evidence, request course context, route each turn, validate handoffs, and maintain the trace. It does not diagnose mastery or silently repair student work. |
| Learner state | Maintain an evidence-backed account of what the student has demonstrated, misconceptions, prerequisites, uncertainty, assistance, and unresolved needs. It does not select the next intervention. |
| Tutoring decision | Combine learner state, Student Evidence, Course Context, Question Schema, retrieved material, and tutoring policy to select the next question, hint, explanation, retry, comprehension check, or transfer action. |
| Tutoring delivery | Present the approved action through text, equations, avatar, speech, flashcards, or another channel without changing its pedagogical intent. |
| System configuration | Maintain course policies, tutoring rules, tool permissions, privacy settings, and delivery preferences that other modules require but do not produce. |
| Persistent state and trace | Store document versions, sessions, evidence, retrieval events, learner states, actions, and delivery history while preserving links and provenance. |

### 3.1 Structured Course Context and Question Schema

| Record | Core contents | Direct consumers |
|---|---|---|
| `CourseContext` | Course identity and version, term, overview, learning objectives, topic hierarchy, prerequisite graph, policies, assessment structure, and source references | RAG uses scope and filters; interaction identifies the course and concept; learner state uses objectives and prerequisites; tutoring decision applies policies; delivery displays course-aware labels and guidance. |
| `QuestionSchema` | Question ID and version, topic/objective links, prompt, expected response type, rubric or success criteria, difficulty, prerequisites, permitted assistance, source references, and delivery hints | Interaction binds evidence to the question; learner state compares evidence with expected work; tutoring decision chooses allowed support; delivery renders the prompt, response controls, feedback, or alternate channel. |

These records contain verified structure rather than retrieved prose. A downstream module may consume them directly even when no RAG call is needed. Retrieved passages remain separately attributed supporting material.

### 3.2 Main logical functions

These names describe system responsibilities and do not require a particular language or service structure.

| Function | Purpose |
|---|---|
| `register_document` | Store the original artifact, create its identity and version, and start a processing job. |
| `classify_document` | Determine the document type and return confidence and alternatives. |
| `extract_structure` | Recover pages, headings, paragraphs, equations, tables, and document-specific fields. |
| `build_course_context` | Create the verified course, topic, learning-objective, prerequisite, policy, and source-reference structure consumed across the system. |
| `validate_question_schema` | Check a question's identity, concept, prompt, expected response, rubric or criteria, difficulty, prerequisite links, permitted support, and source references. |
| `build_chunks` | Create retrievable units with stable source locations and parent-child relationships. |
| `index_course_content` | Write chunks to vector and keyword indexes using the current embedding and index version. |
| `retrieve_course_context` | Apply course and document filters, retrieve candidates, rerank them, and return cited passages. |
| `receive_submission` | Preserve student input and attach session, problem, modality, and turn context. |
| `confirm_recognition` | Ask the student to resolve recognition alternatives that could change meaning. |
| `build_student_evidence` | Represent the student's confirmed work, explanation, uncertainty, revision, and prior assistance. |
| `update_learner_state` | Convert validated evidence into an evidence-backed state update without choosing an intervention. |
| `select_tutoring_action` | Apply the question schema, learner state, course context, retrieved passages, and tutoring rules to choose the next action. |
| `route_tutoring_turn` | Send validated evidence and course context to the correct state, decision, retrieval, and delivery components. |
| `deliver_action` | Render an approved tutoring action and record what the student actually received. |
| `record_turn_trace` | Link submissions, evidence, retrieval, decisions, tool use, delivery, and responses. |

# 4 Key inputs and outputs

This table states the main contract for each module. Detailed field definitions appear later in the database and handoff sections.

| Module | Receives | Returns |
|---|---|---|
| Course documents | Uploaded source files, course association, and approval or replacement decision | Stable document/version IDs, original-artifact references, metadata, and availability status |
| Document extraction | Approved document version and extraction/chunking policy | Classification, structured pages, extracted fields, chunks, source locations, confidence, and warnings |
| Course Context / Question Schema | Verified extracted course fields, topic structure, authored problems, policies, and source references | Versioned `CourseContext` and `QuestionSchema` records with validation status |
| Course knowledge/RAG | Extracted chunks, index configuration, live query, course scope, and filters | Index status or ranked passages, parent context, page citations, scores, and insufficient-context status |
| Student interaction | Raw student input, channel capabilities, and current turn context | Submission, recognition candidates, uncertainty, confirmation status, and student response |
| Interaction integration | Submission, recognition, prior action, session context, and retrieved passages | Student Evidence, grounded turn context, routing request, and complete trace |
| Learner state | Student Evidence, prior state, Course Context, Question Schema, and retrieved context | Updated evidence-backed state, unresolved needs, and confidence/status |
| Tutoring decision | Learner state, Student Evidence, Course Context, Question Schema, retrieved passages, tutoring rules, and tools | Selected action, goal, constraints, sources, allowed support, and expected response |
| Tutoring delivery | Approved action, Course Context/Question Schema display fields, channel capabilities, and preferences | Presented problem or intervention, delivery status, and next-response link |
| System configuration | Course policies, team-approved tutoring rules, permissions, retention policy, and user preferences | Versioned constraints, available-tool list, and channel settings |
| Persistent state and trace | Validated records and events from every module | Prior session/learner state, linked history, active versions, and audit trace |

## 5 Handoffs between components

The system passes named records rather than loose text. The table below shows the direction and purpose of each major handoff.

| Sender | Handoff | Receiver | Purpose |
|---|---|---|---|
| Course documents | Document/version record | Document extraction | Identify the approved, immutable source to process. |
| Document extraction | `StructuredDocument` | Course Context/Question Schema and Course knowledge/RAG | Supply classified pages, hierarchy, chunks, fields, equations, source locations, and warnings. |
| Course Context / Question Schema | `CourseContext` and `QuestionSchema` | RAG, interaction integration, learner state, tutoring decision, and delivery | Give every downstream module the same course, concept, problem, prerequisite, expected-response, rubric, policy, and source-reference meaning. |
| Interaction integration | `RetrievalRequest` | Course knowledge/RAG | Ask for course material scoped to the current turn. |
| Course knowledge/RAG | `RetrievedContext` | Interaction integration, learner state, and tutoring decision | Ground the turn with ranked passages and page citations, or report insufficient context. |
| Student interaction | `StudentSubmission` and `RecognitionResult` | Interaction integration | Supply original input, interpreted content, alternatives, uncertainty, and confirmation status. |
| Interaction integration | `StudentEvidence` and grounded turn | Learner state | Supply confirmed work, explanation, uncertainty, revisions, prior assistance, course/question identity, and retrieved sources. |
| Learner state | `LearnerState` | Tutoring decision | State what the evidence supports, assistance already used, unresolved needs, confidence, and prerequisite or misconception flags. |
| Tutoring decision | `TutoringAction` | Tutoring delivery | Specify the next approved action, goal, constraints, sources, permitted help, and expected response. |
| Tutoring delivery | `DeliveryResult` | Interaction integration | Record what the student received and connect the next response to that action. |
| System configuration | Policy and settings record | Interaction integration, learner state, tutoring decision, and delivery | Supply rules, permissions, tools, privacy constraints, and channel settings that are not produced by the preceding module. |
| Persistent state and trace | Prior-state and history records | Interaction integration, learner state, and tutoring decision | Restore prior state and accepted versions; receive validated records from all modules for storage. |

Every handoff preserves its origin, status, and version. Student-authored content, recognition output, retrieved material, system interpretation, and tool output remain distinguishable.

# 6 Supporting note: document extraction and RAG design

This section records useful implementation-facing detail, but it is supporting material for the current milestone. The immediate design decision is the boundary between structured course/question context, searchable course knowledge, and their downstream consumers.

## 6.1 Ingestion pipeline

1. **Register and preserve.** Store the original file, calculate a checksum, assign document and version identifiers, and associate it with a course only when that association is known.
2. **Classify.** Identify the material as a syllabus, lecture notes, textbook, assignment, worked example, or other type. Preserve confidence and alternate classifications.
3. **Extract.** Recover page text and layout. Apply OCR to image-only pages. Preserve equations, tables, headings, lists, page numbers, and reading order rather than flattening everything into one text block.
4. **Normalize.** Remove repeated headers and footers, repair safe line-break artifacts, retain meaningful notation, and record transformations. Keep the raw extraction for comparison.
5. **Structure.** Build a document hierarchy such as document, chapter, section, subsection, page, and content block. Extract syllabus-specific course fields without mixing those fields into lecture-note topic extraction.
6. **Chunk.** Create retrieval units from the structure, attach complete metadata, and retain links to the larger parent section and original page.
7. **Index.** Create embeddings, keyword-search entries, and filters for course, document type, document version, topic, section, and page.
8. **Validate and publish.** Check extraction coverage, empty or duplicate chunks, missing page links, broken equations, and index counts. Only approved document versions become available to retrieval.

## 6.2 Chunking strategy

Chunking should follow document meaning before token count. The initial policy should support separate rules by material type:

- **Syllabi:** use semantic sections such as overview, learning outcomes, schedule, assessment, and policies. Store structured fields separately and link each field to its source chunk and page.
- **Lecture notes:** chunk by heading and coherent explanation or worked step. Keep a theorem with its conditions, an equation with its definitions, and a worked example with the steps needed to interpret it.
- **Textbooks:** use a parent-child design. Small child chunks support precise retrieval; larger section or subsection parents restore context after retrieval.
- **Equations and tables:** store a normalized representation plus nearby explanatory text and the original page region. Do not separate symbols from their definitions.

- **Size and overlap:** begin with moderate child chunks, roughly 300–700 tokens, and small overlap only where a boundary divides a continuing explanation. Measure the actual distribution instead of forcing every chunk to the same size.
- **Traceability:** every chunk retains course, document, version, type, hierarchy path, page range, character or block offsets, and parent identifiers.

### 6.3 Setting up retrieval for success

- Index only approved document versions and prevent cross-course retrieval by default.
- Combine semantic vector search with keyword or lexical search so exact course terms, theorem names, symbols, and policy wording remain findable.
- Apply metadata filters before or during retrieval, including course, term, document type, topic, and active version.
- Retrieve a broad candidate set, rerank it for the current question, and return a small set of passages with page-level citations.
- Expand a selected child chunk with its parent or neighboring chunks when the local passage lacks definitions or necessary steps.
- Preserve the retrieval query, filters, candidate scores, reranking result, index version, and returned chunk IDs in the turn trace.
- Return an explicit insufficient-context result when the knowledge base does not support the request. The tutoring model must not invent course-specific claims.
- Re-index deterministically when parsing, chunking, embedding, or source versions change; keep old traces tied to the index version that produced them.

## 7 Supporting note: persistent data and database schemas

This section preserves a possible persistence model for later refinement. The current milestone requires agreement on record meaning and ownership; exact tables, field types, and storage technologies can remain provisional.

A relational database should hold identity, state, provenance, and relationships. A vector index should hold embeddings keyed to relational chunk IDs. Original artifacts belong in file or object storage. These may be deployed together for a prototype, but their logical responsibilities should remain separate.

| Table or store | Core fields | Purpose and relationships |
|---|---|---|
| `courses` | `course_id`, name, number, term, overview, status | Root scope for documents, sessions, retrieval, and course configuration. |
| `course_contexts` | context ID, course/version IDs, objectives, topic and prerequisite graph, policies, assessment structure, source links, status | Stable structured course meaning shared directly with interaction, learner state, tutoring decisions, delivery, and retrieval. |

| Table or store | Core fields | Purpose and relationships |
|---|---|---|
| `question_schemas` | question ID, context/topic IDs, prompt, expected response type, rubric or criteria, difficulty, prerequisites, permitted support, source links, version | Defines a question independently of any one delivery channel or student attempt. |
| `documents` | `document_id`, course ID, type, title, source URI, checksum, active version | Stable identity for each syllabus, note set, textbook, assignment, or example. |
| `document_versions` | `version_id`, document ID, file URI, parser version, status, created time | Keeps ingestion reproducible and prevents a replaced file from changing old traces. |
| `pages` | `page_id`, version ID, page number, raw text, normalized text, image URI | Page-level source and display unit. |
| `sections` | `section_id`, version ID, parent section ID, heading, level, page range | Document hierarchy used by chunking and context expansion. |
| `chunks` | `chunk_id`, version, section and page IDs, text, token count, offsets, parent chunk ID, metadata | Canonical retrievable unit and citation target. |
| Vector index | `chunk_id`, embedding, embedding model/version, searchable metadata | Semantic retrieval; the chunk ID resolves to authoritative content in the relational store. |
| Keyword index | `chunk_id`, indexed text and fields | Exact-term and lexical retrieval used alongside vector search. |
| `ingestion_jobs` | job ID, version ID, stage, status, warnings, metrics, start/end time | Tracks extraction, chunking, indexing, failures, and approval. |
| `retrieval_events` | retrieval ID, turn ID, query, filters, index version, candidate and returned chunk IDs, scores | Makes every grounded response reproducible and inspectable. |
| `sessions` and `turns` | session/turn IDs, course ID, student reference, problem context, times, status | Orders the interaction and connects it to the correct course scope. |
| `submissions` | submission ID, turn ID, modality, raw artifact URI, entered text, timestamp | Immutable record of what the student provided. |
| `recognition_results` | result ID, submission ID, candidate content, alternatives, uncertainty, provider/settings | Separates machine recognition from confirmed student content. |
| `student_evidence` | evidence ID, turn ID, confirmed content, steps, explanation, uncertainty, prior action, revision link, status | Canonical evidence passed to learner state and tutoring decision. |
| `learner_states` | state ID, session ID, concept statuses, misconceptions, prerequisites, evidence links, version | Evidence-backed state at a point in the interaction. |
| `tutoring_actions` | action ID, turn ID, type, goal, content constraints, source and tool links, expected response | Decision output independent of presentation channel. |
| `delivery_events` | delivery ID, action ID, channel, rendered content, status, timing | Records what the student actually saw or heard. |

All stored records need stable IDs, timestamps, producer or source, status, and schema or model version where applicable. Foreign keys should enforce course, document, session, turn, and revision relationships. Deleting or replacing a derived record must not erase the original course file or student submission.

# 8 Complete workflow: what enters, leaves, and is tracked

## 8.1 Course setup workflow

| Stage | Returns | Tracks |
|---|---|---|
| Register course document | Document and version IDs; stored-artifact reference; processing status | Checksum, storage location, uploader, course association, time, and approval state |
| Classify and extract | Document type; pages; hierarchy; equations; tables; extracted course fields | Parser/OCR versions, confidence, alternatives, warnings, and transformations |
| Build structured course/question context | Validated `CourseContext` and `QuestionSchema` records | Topic and prerequisite links, expected response, rubric/criteria, difficulty, policies, permitted support, sources, author, version, and validation status |
| Build chunks | Linked chunks ready for indexing | Section hierarchy, page range, offsets, token count, overlap, parent links, and chunking-policy version |
| Build indexes | Vector and keyword entries; index version; indexing status | Embedding model/version, rejected chunks, searchable metadata, and completion counts |
| Publish course knowledge | Retrieval-ready and coverage status | Active document/index versions and missing, failed, or superseded material |

### 8.2 Tutoring-turn workflow

| Stage | Returns | Tracks |
|---|---|---|
| Capture student input | Accepted submission and immutable artifact reference | Session, problem, modality, turn, timestamp, and current delivery/action link |
| Recognize and confirm | Candidate content, alternatives, uncertainty, and confirmed interpretation | Provider, settings, latency, candidate history, and student correction effort |
| Build Student Evidence | Validated work, steps, explanation, uncertainty, and revision | Provenance, prior assistance, attempt/revision links, and validation status |
| Apply course/question context | Turn context with stable course, concept, question, prerequisite, expected-response, rubric, and source identities | Context/schema versions and which fields were supplied to each downstream module |
| Retrieve course context | Ranked course passages or insufficient-context status | Query, filters, index version, candidates, scores, selected chunks, and citations |
| Update learner state | Evidence-backed current state | Evidence supporting each conclusion, misconceptions, prerequisites, help, and unresolved issues |
| Select tutoring action | One approved question, hint, explanation, retry, check, or transfer action | Pedagogical goal, constraints, sources, tools, rules, and expected response |
| Deliver action | Presented content and delivery status | Exact content, channel, timing, interruptions, preferences, and action link |
| Capture next response | New submission and turn linked to the action | Full path from course sources and evidence through decision, delivery, and revision |

### 8.3 Learner state to tutoring action

The learner-state record describes the student without prescribing the intervention. The tutoring-decision component combines that state with the current Question Schema, Course Context, retrieved passages, and tutoring policy. This separation allows decision rules to change without rewriting the evidence history.

| Learner-state signal | Question-schema constraint | Possible effect on Tutoring Action |
|---|---|---|
| Evidence is sufficient and correct | Required reasoning and response type are satisfied | Confirm briefly, then choose a comprehension or transfer check. |
| Partially correct step or local misconception | Rubric identifies the missing criterion or step | Ask a targeted question or give a bounded hint tied to that criterion. |
| Wrong approach | Expected method or concept relationship conflicts with the attempt | Surface the mismatch and request a revised approach without silently solving the task. |
| Missing prerequisite | Question lists a prerequisite not yet demonstrated | Run a prerequisite check or short review before returning to the current question. |
| Insufficient evidence | Expected response requires reasoning, work, or an explanation that is absent | Request the missing work rather than infer mastery from an answer alone. |
| High uncertainty or repeated difficulty | Assistance limit, retry count, or escalation rule is reached | Change strategy, reduce step size, explain more directly, or request human review. |

# 9 Things to agree on

- The final module list, the owner of each module, and the owner of every handoff.
- Which document types enter the first course knowledge base and who approves a document version.
- The extraction output required for syllabi, lecture notes, textbooks, equations, and tables.
- The minimum contents, ownership, authoring process, and validation rules for `CourseContext` and `QuestionSchema`.
- Which structured course/question fields RAG, interaction integration, learner state, tutoring decision, and delivery each consume directly.
- Initial chunking policies by document type, including hierarchy, size, overlap, and parent expansion.
- Vector, keyword, and reranking responsibilities and the minimum metadata required for filtering and citation.
- The minimum fields in each database table and handoff record, plus versioning and retention rules.
- Where Student Evidence ends, what learner state may infer, and what information tutoring decision must use before selecting an action.
- When recognition ambiguity must return to the student for confirmation.
- What each component returns when extraction, retrieval, recognition, a tool, or delivery fails.

## 10 Things to do and proposed collaboration

1. **Ovunc and Sean:** define the syllabus and lecture ingestion path from uploaded file through structured documents, structured course/question context, chunks, indexes, and retrieval-ready status.
2. **Ovunc:** define `CourseContext` and `QuestionSchema`, including topics, prerequisites, learning objectives, prompt, expected response, rubric/criteria, difficulty, policies, permitted support, and source references.
3. **Sean:** specify chunking, embeddings, hybrid retrieval, reranking, filters, citations, index versioning, and the knowledge-base database needs.
4. **Sean and Jonathan:** agree on what retrieval receives from structured context and a student turn, and what grounded context returns to Student Evidence, learner state, and tutoring decision.
5. **Ovunc and Jonathan:** define how Question Schema and Student Evidence combine to update learner state, then specify the learner-state-to-tutoring-action contract.
6. **Jonathan:** define Student Evidence, interaction routing, trace records, learner-state updates, and the handoffs connecting recognition, structured context, retrieval, decisions, and delivery.
7. **Ovunc and Yining Li:** identify the Course Context and Question Schema fields delivery needs to render questions, response controls, feedback, flashcards, speech, and avatar behavior consistently.
8. **Yining Li:** define how a selected tutoring action is expressed through each channel and how the student's response returns to interaction integration.
9. **Whole team:** review the database schema and one complete course-setup and tutoring-turn walkthrough; resolve missing fields, duplicated responsibility, and unclear failure handling.
10. Freeze architecture and interface version 0.1 before choosing detailed APIs or implementing deeper module internals.

## 11 Future work

After architecture and contracts stabilize, the team can:

- create a small Calculus tutoring workflow and example set to validate the integrated design;
- define formal evaluation criteria for extraction, retrieval, Student Evidence, grounding, tutoring decisions, and delivery;
- compare chunking, embedding, retrieval, reranking, recognition, and model alternatives;
- measure retrieval quality, response reliability, latency, cost, and correction effort;
- expand course-specific workflows while keeping the shared architecture course-agnostic; and
- implement broader avatar and real-time interaction support.

The present milestone is complete when the team can trace an uploaded course document into structured Course Context, Question Schema, and the searchable knowledge base, then trace a student turn through evidence, learner state, tutoring action, and delivery, with a clear owner, input, output, stored record, and failure result at every step.